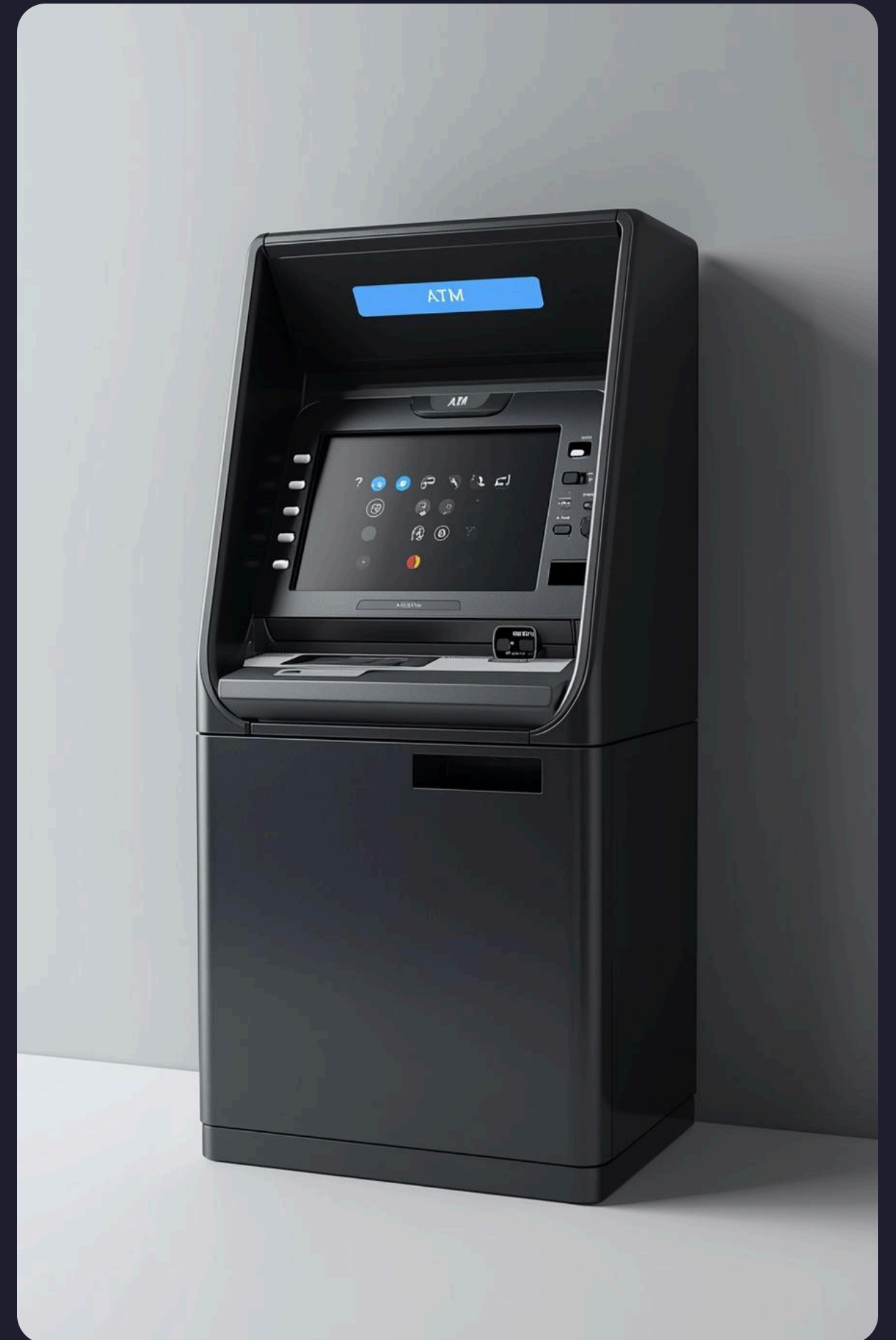


ATM Simulation System



Project Overview :

The project simulates the operation of a real ATM, allowing the user to log in using a PIN and perform banking transactions such as cash withdrawal and balance inquiry.



Project Objectives :

- Applying Object-Oriented Programming (OOP) concepts in a real-world project.
- Managing user data (account, balance, and transactions).
- Simulating the banking user interface.

Introduction :

The project is a simplified model of an ATM system that allows users to perform a set of basic banking operations. The system is designed using Object-Oriented Programming (OOP) principles and focuses on simulating the user experience when interacting with ATMs through a simple and clear interface.

OOP Concepts : (Encapsulation)

Using private variables(name,ID)with getters and setter to protect data and allow secure access to it

```
2
3 | private final int accountNumber;
4   private double balance;
5
6   [-] public Account(int accountNumber, double balance) {
7       this.accountNumber = accountNumber;
8       this.balance = balance;
9   }
10
11   // getters / setters (encapsulation)
12   [-] public int getAccountNumber() {
13       return accountNumber;
14   }
15
16   [-] public double getBalance() {
17       return balance;
18   }
19
20   [-] public void setBalance(double balance) {
21       this.balance = balance;
22   }
```

Abstraction:

Abstraction was used through the ATMOperations interface, which defines the ATM functions (viewBalance, withdraw, deposit) without showing how they are implemented. The actual implementation is done later in the ATMMachine class.

```
public interface ATMOperations {  
    void viewBalance(Account acc);  
    void withdraw(Account acc, double amount);  
    void deposit(Account acc, double amount);  
}
```

Inheritance:

The ability to create a new class that extends an existing class allows for the inheritance of its properties and behaviors in the code. Inheritance was used when the ATMMachine class extended the Person class, allowing it to inherit the name and id attributes and the displayInfo method, which it then overridden to fit the ATM's functionality.

```
public class ATMMachine extends Person implements ATMOperations {  
  
    public ATMMachine(String name, int id) {  
        super(name, id);  
    }  
}
```

Polymorphism

Polymorphism enables a method to exhibit varying implementations across different classes. In the provided code, the `displayInfo()` method is overridden in the `Client` class to present client-specific information.

```
public void displayInfo() {  
    System.out.println("ID: " + id + ", Name: " + name);  
}
```

```
@Override  
public void displayInfo() {  
    System.out.println("Client: " + name + " | Balance: " + balance);  
}
```

Person Class:

Represents a general person in the system.

It contains basic information such as the name and ID, and serves as a base for other classes like Client.

```
class Person {  
    protected String id;  
    protected String name;  
  
    public Person(String id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public void displayInfo() {  
        System.out.println("ID: " + id + ", Name:");  
    }  
    public String getName() {  
        return name;  
    }  
    public String getId() {  
        return id;  
    }  
}
```

ATMMachine class:

Inherits from person class
implements ATMOperations
Overrides the
displayInfo()method
Implements ATM functions:
view balances ,
Deposit,withdraw.

```
*/  
public class ATMMachine extends Person implements ATMOperations {  
  
    public ATMMachine(String name, int id) {  
        super(name, id);  
    }  
  
    @Override  
    public void displayInfo() {  
        System.out.println("ATM Machine Handled By: " + name + " | ID: " + id);  
    }  
  
    @Override  
    public void viewBalance(Account acc) {  
        System.out.println("Balance: " + acc.getBalance());  
    }  
  
    @Override  
    public void withdraw(Account acc, double amount) {  
        if (amount <= acc.getBalance()) {  
            acc.setBalance(acc.getBalance() - amount);  
            System.out.println("Withdrawn: " + amount);  
        } else {  
            System.out.println("Insufficient Balance!");  
        }  
    }  
  
    @Override  
    public void deposit(Account acc, double amount) {  
        acc.setBalance(acc.getBalance() + amount);  
        System.out.println("Deposited: " + amount);  
    }  
}
```

Main Class :

- The starting point of the program.
- Adds demo clients, then interacts with the user through the list
- Allows the customer to choose operations such as balance verification, deposit, withdrawal, or exit the system.

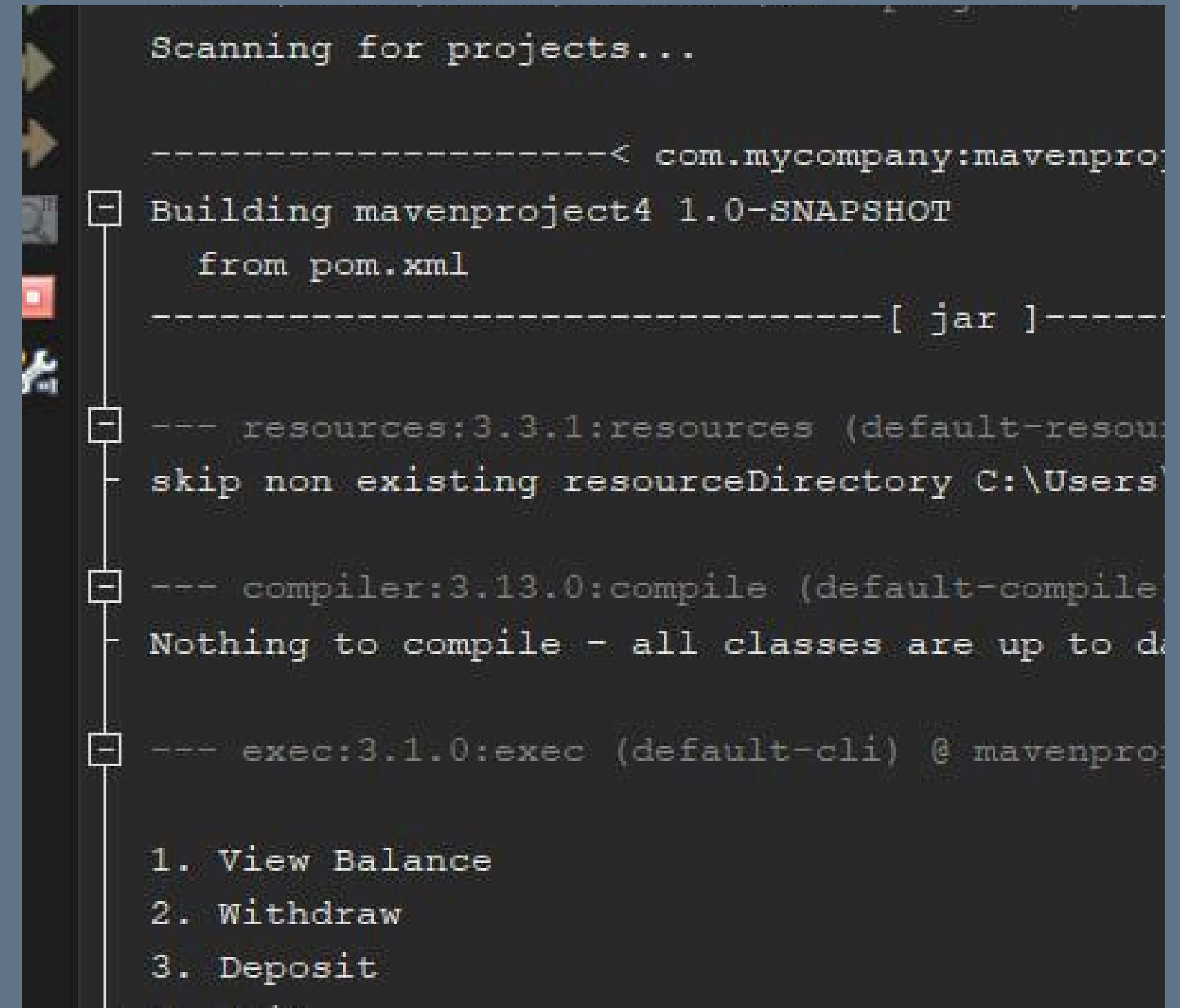
```
3 public class Main {
4     public static void main(String[] args) {
5
6         Account acc = new Account(12345, 2000);
7
8         // Polymorphism
9         ATMOperations atm = new ATMMachine("Manar", 101);
10
11         Scanner in = new Scanner(System.in);
12
13         OUTER:
14         while (true) {
15             System.out.println("\n1. View Balance\n2. Withdraw\n3. Deposit\n4. Exit");
16             System.out.print("Enter Choice: ");
17             int ch = in.nextInt();
18             switch (ch) {
19                 case 1 -> atm.viewBalance(acc);
20                 case 2 -> {
21                     System.out.print("Withdraw Amount: ");
22                     atm.withdraw(acc, in.nextDouble());
23                 }
24                 case 3 -> {
25                     System.out.print("Deposit Amount: ");
26                     atm.deposit(acc, in.nextDouble());
27                 }
28                 case 4 -> {
29                     System.out.println("Thank you for using ATM!");
30                     break OUTER;
31                 }
32                 default -> {
33                 }
34             }
35         }
36     }
37 }
```

Output:

System Flow:

1. The program initiates.
2. The user selects an option from the menu.
3. The chosen operation is executed.
4. The account balance is updated.

The process continues until the user selects "Exit."



```
Scanning for projects...

-----< com.mycompany:mavenproj
Building mavenproject4 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----

--- resources:3.3.1:resources (default-resou
skip non existing resourceDirectory C:\Users\

--- compiler:3.13.0:compile (default-compile
Nothing to compile - all classes are up to d

--- exec:3.1.0:exec (default-cli) @ mavenproj

1. View Balance
2. Withdraw
3. Deposit
4. Exit
```

The screenshot shows a terminal window with Maven build output. The output includes 'Scanning for projects...', project details for 'com.mycompany:mavenproj', and build phases like 'resources', 'compile', and 'exec'. At the bottom, a menu is displayed with options: '1. View Balance', '2. Withdraw', '3. Deposit', and '4. Exit'. A white arrow on the right side of the terminal points upwards towards the menu.

By:

1. Almaha Abdulrahman Alzubaidi
2. Alraha Abdullah Alzubaidi
3. Manar Laheq Alzubaidi
4. Sarah Ali Alamri

